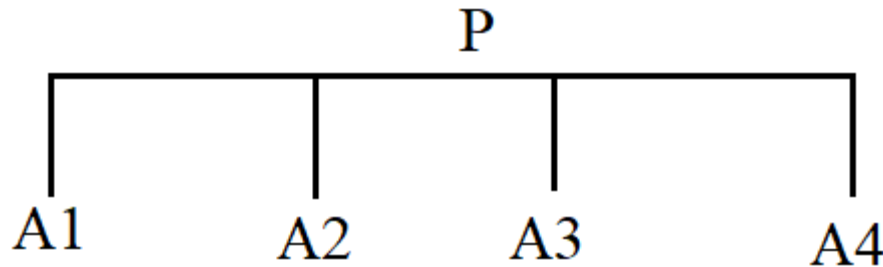


Algorithm

Algorithms

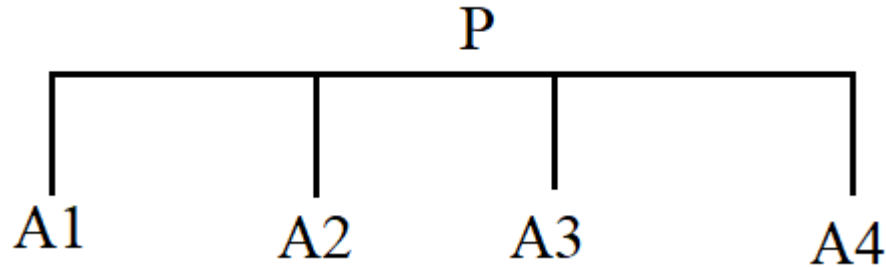
Suppose we want to implement a problem P in any programming language. So, before writing the program, we first design algorithm(s).

There may be more than one algorithms for a given problem.



How to decide which is the best algorithm among all? What are the different parameters that helps us to compare algorithms?

Algorithms



we can compare the algorithm based on **time** and **memory (space) functions**.

In this course we will discuss how to **design different algorithms** for a **problem** and then **analysing** and **selecting** the **best one** for the implementation.

Let **first discuss Algorithm** followed by the **different ways used for analysis**.

Algorithms

What is an Algorithm?

What are the properties of an Algorithm?

Algorithms

What is an Algorithm?

- It is a combination, or a sequence of finite steps required to solve a given problem.

What are the properties of an Algorithm?

- It should produce **at least one output**;
- It should not have any **ambiguous statements**;
- It should produce **an output** within a **finite amount of time**;
- It should take a **finite number of inputs**.
- It should be **loop invariant**.
- It should be **language independent**.
- Correctness: It should produce the **same output** for the **given same input** at any given time.

Why study algorithms?

- Algorithms help us to understand the **basic idea of the problem**.
- To find **different approaches** to **solve the problem**.
- The algorithm gives a **clear description** of requirements and **goal of the problem**.
- With the help of an algorithm, we can also identify the resources **(memory, input-output) cycles** required by the algorithm.
- To measure the behavior (or performance) of the methods in all cases (**best cases, worst cases, average cases**).
- We can measure and analyse the **complexity (time and space) of the problems** concerning **input size** without **implementing and running it**; it will **reduce the cost of design**.

Algorithm vs Program

Algorithm	Program
Design dependent	Implementation dependent
Requires domain knowledge	Programmer
Any language or mathematical notations can be used for writing algorithm	Programming language
H/W and S/W independent	Dependent on H/W and S/W
Analyze the algorithm for desired result	Test the program for the correct result
Algorithm requires priori analysis	Programs need posteriori testing

How to Analyse Algorithms

Main Aspects

- **Time**
 - Need to define a time function to determine the time taken by the algorithm
- **Space**
 - Need to define a space function to determine the space required for the algorithm

Other Aspects

- Network consumption
- Power consumption

Reasons to analyse algorithms

- Predict performance.
- Compare algorithms.
- Provide guarantees.
- Understand theoretical basis.

Primary practical reason: avoid performance bugs.



**client gets poor performance because programmer
did not understand performance characteristics**



How to Analyse Algorithms

```
Algorithm swap(a, b)
{
    temp=a;
    a=b;
    b=temp;
}
```

Time: ?

Space: ?

How to Analyse Algorithms

Algorithm swap(a, b)

{

temp=a; ----- 1 unit

a=b; ----- 1 unit

b=temp; ----- 1 unit

}

$f(n)=3$ [time function]

Time

It means this algorithm will require **constant time** to return the output.

Space:

Number of variables in the algorithm are **temp, a, and b**. So, space complexity will be $S(n)=1+1+1=3$ word and **it is also constant**.

$X = 10*a + 2*c + 8/2$, Time =? Space=?

Analyze algorithms: Frequency count method

Algorithm sum(A, n) // A is an array

```
{
    s=0; ----- 1
    for(i=0; i<n; i++) --- n+1 (1+n+1+n=2n+2)
    {
        s=s+ A[i]; --- n
    }
    return s; ----- 1
}
```

$f(n) = 2n+3$, as degree of polynomial is 1 so it could be written as $O(n)$.

Space complexity

No. of variables are 4 i.e., A, n, i, s , where size of A is n words and others are 1 so, space complexity will be $n+3$ i.e., $O(n)$.

Another Example

Algorithm Add (A, B, n)

```
{  
    for (i=0; i<n; i++)  
    {  
        for (j=0; j<n; j++) {  
            C[i, j] = A[i, j] + B[i, j];  
        }  
    }  
}
```

Time: ?

Space: ?

Another Example

Algorithm Add (A, B, n)

```
{
    for (i=0; i<n; i++) ----- n+1
    {
        for (j=0; j<n; j++) ----- n* (n+1)
        {
            C[i, j] = A[i, j] + B[i, j]; -- n*n
        }
    }
}
```

Highest degree is **2**, so it could be

written as **$O(n^2)$**

$$f(n) = 2*n^2 + 2n + 1$$

Space complexity

Variables are **A, B, C, n, i, j**, where **A, B, and C** are matrices so, space required for them will be **n^2** and for **n, i, and j** it will **1 words**. Total space required will be **$3n^2+3$** words equivalent to **$O(n^2)$** .

Another Example

```
Algorithm multi (A, B, n)
{
    for (i=0; i<n; i++)
    {
        for (j=0; j<n; j++)
        {
            c[i, j]=0;
            for (k=0; k<n; k++)
            {
                c[i, j] = c[i, j] + A[i, k]*B[k, j];
            }
        }
    }
}
```

Time: ?

Space: ?

Another Example

Algorithm multi (A, B, n)

```
{
    for (i=0; i<n; i++) ----- n+1
    {
        for (j=0; j<n; j++) ----- n* (n+1)
        {
            c[i, j]=0; ----- n*n
            for (k=0; k<n; k++) ----- n*n* (n+1)
            {
                c[i, j]= c[i, j]+ A[i, k]=B[k, j]; ---- n*n*n
            }
        }
    }
}
```

$f(n) = 2n^3 + 3n^2 + 2n + 1$
 $= O(n^3)$

Space complexity

Variables are **A, B, C, n, i, j, k** where **A, B, and C** are matrices so, space required for them will be **n^2** and for **n, i, j and k, it will 1 words**. Total space required will be **$3n^2+4$** words equivalent to **$O(n^2)$** .

Another Example

```
for (i=0; i<n; i++)  
{  
    for (j=0; j<i; j++)  
    {  
        Stmt;  
    }  
}
```

Time: ?

Another Example

```
for(i=0; i<=n; i++)  
{  
    for(j=0; j<i; j++)  
    {  
        Stmt;  
    }  
}
```

<u>i</u>	<u>j</u>	<u>no. of time</u>
0	0	0 time
1	0/1	1
2	0,1/2	2
3	0,1,2/3	3
.		
n	0,1,..n/n+1	n times

It will run in total $1+2+3+\dots+n$
 $\Rightarrow n(n+1)/2 \Rightarrow O(n^2)$

Another Example

1. `for(i=0; i<n; i++)` --- **$n+1$**
 {
 `Stmt;` --- **n**
 }
 $f(n) = O(n)$

2. `for(i=n; i>0; i--)` --- **$n+1$**
 {
 `Stmt;` --- **n**
 }
 $f(n) = O(n)$

3. `for(i=1; i<n; i+2)` --- **$n/2+1$**
 {
 `Stmt;` --- **$n/2$**
 }
 $f(n) = O(n)$

Example

```
5.  p=0;  
    for (i=1; p<=n; i++)  
    {  
        p=p+i;  
    }
```

Time: ?

Example

```
5. p=0;
   for (i=1; p<=n; i++)
   {
       p=p+i;
   }
```

<u>i</u>	<u>p</u>
1	0+1
2	0+1+2
3	0+1+2+3
.	.
k	0+1+2+...+k

Assume it will stop when $p > n$,

As $p = 1+2+3+\dots+k = \frac{k(k+1)}{2}$

So, $\frac{k(k+1)}{2} > n$

$= \frac{k^2}{2} > n$

$k > \sqrt{2n}$

This for loop will execute for $O(\sqrt{n})$ times.

Example

```
6.  for (i=1; i<n; i=i*2)
    {
        stmt;
    }
```

Time: ?

Example

```
6.  for (i=1; i<n; i=i*2)
      {
          stmt;
      }
```

i

1

$1*2=2$

$2*2=2^2$

$2^2*2=2^3$

.

.

$2^{k-1}*2=2^k$

This will stop when $i \geq n$

As, $i=2^k$

So, $2^k > n \Rightarrow k * \log_2 2 = \log_2 n$

It means stmt will execute $\log_2 n$ times

k = O (ceil(log₂n))

n=8

i

1<8 Y

2<8 Y

4<8 Y

8<8 N

3 times

$\log_2 8 = 3$

$\log_2 2^3 = 3$

**must be 4. therefore, we will take
ceil(log 10)**

n=10

i

1

2

4

8

4 times

$16 > 10$ N

$\log 10 = 3.2 < 4$

Since, it

must be 4. therefore, we will take

ceil(log 10)

Observation from Analysis

I. `for (i=0; i<n; i++)` $\rightarrow O(n)$

II. `for (i=0; i<n; i+2)` $\rightarrow n/2 = O(n)$

III. `for (i=n; i>1; i--)` $\rightarrow O(n)$

IV. `for (i=1; i<n; i*2)` $\rightarrow O(\log_2 n)$

V. `for (i=1; i<n; i*3)` $\rightarrow O(\log_3 n)$

VI. `for (i=n; i>1; i=i/2)` $\rightarrow O(\log_2 n)$

Exercise-1

```
1. for (i=1; i>=1; i=i/2)
{
    Stmt;
}
```

```
2. for (i=1; i<n; i=i*2)
{
    p++;
}
for (j=1; j<p; j=j/2)
{
    stmt;
}
```

```
3. for(i=0; i*i<n; i++)
{
    stmt;
}
```

```
4. for (i=0; i<n; i++)
{
    for(j=1; j<n; j*j<n)
    {
        stmt;
    }
}
```

Analysis of if and while

```
1. i=0 -----1
   while (i<n) -----n+1
   {
       stmt; -----n
       i++; -----n
   }
```

Total time taken= **$f(n) = 3n+2$**
 $= O(n)$

Analysis of if and while

(i)

```
a=1
while (a<b)
{
    stmt;
    a=a*2;
}
```

Time: ?

(ii)

```
i=1;
k=1;
while (k<n)
{
    stmt;
    k=k+i;
    i++;
}
```

Time: ?

Analysis of if and while

```
a=1
while (a<b)
{
    stmt;
    a=a*2;
}
```

a
1
1*2
1*2*2
1*2*2*2
.
 2^k

This will terminate if $a \geq b$

As $a = 2^k$ so, it should be greater than b.

$2^k \geq b \Rightarrow k = \log_2 b$

$k = O(\log_2 b)$

Analysis of if and while

```
i=1;  
k=1;  
while (k<n)  
{  
    stmt;  
    k=k+i;  
    i++;  
}
```

i	k
1	1
2	1+1
3	2+2
4	2+2+3
5	2+2+3+4
.	.
m	2+2+3+4+...+m = $m(m+1)/2$

It will terminate if $k \geq n$. So, $m(m+1)/2 \geq n$
 $m^2 \geq n$

$$\mathbf{m} = \sqrt{n} = O(\sqrt{n})$$

Same can also be written using for

```
for(k=1, i=1; k<n; i++)  
{  
    stmt;  
    k=k+i;  
}
```

Analysis of if and while

Algorithm Test(n)

```
{  
    if (n < 5)  
        printf("%d", n);  
    else  
        for (i = 0; i < n; i++)  
        {  
            printf("%d", i);  
        }  
}
```

Time: ?

Analysis of if and while

Algorithm Test(n)

```
{  
    if(n<5)  
        printf("%d", n);  
    else  
        for(i=0; i<n; i++)  
        {  
            printf("%d", i);  
        }  
}
```

} If n is less 5 than the algo will execute 1 time

} If n is greater 5 than the algo will execute n+1 times

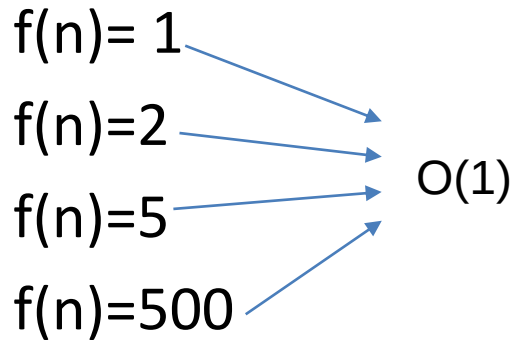
For the above, if statement decides, how many times it will execute. In the above statement if condition will be true then it will execute only once i.e., the best-case time $O(1)$, otherwise it will execute $(n+1)$ times which will be the worst-case time.

Best case= $O(1)$

Worst case= $O(n)$

Types of time functions

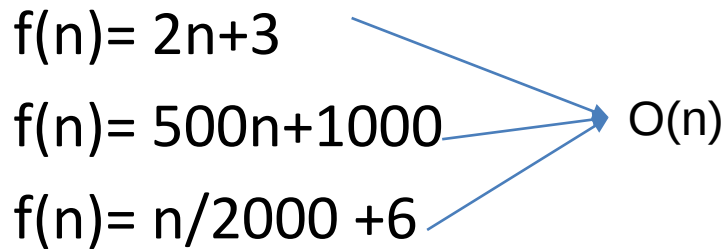
Constant time functions



Logarithmic time functions

$O(\log n)$

Linear time functions



Types of time functions

Quadratic time functions

$$f(n) = O(n^2)$$

Cubic time functions

$$f(n) = O(n^3)$$

Exponential time functions

$$f(n) = O(2^n)$$

$$f(n) = O(3^n)$$

$$f(n) = O(n^n)$$

Comparison of time functions

$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < \dots < n^n$

Log n	n	n^2	2^n
0	1	1	2
1	2	4	4
2	4	16	16
3	8	64	256
4	16	256	65536